PROJECT TECHNICAL REPORT

Flask-to-FastAPI Migration
License Plate Verification System

Framework comparison, request validation, endpoint behavior, database operations, testing workflow, and implementation considerations

PROJECT FILES	app.py · fiftyville.db
TEST CLIENT	Separate test.py script that sends HTTP requests
CURRENT STACK	Python · FastAPI · Pydantic · CS50 SQL · SQLite

Prepared from the group's updated FastAPI implementation

1. Executive Summary

The group has migrated the license plate verification application from Flask to FastAPI. The application's core business logic remains largely unchanged: it receives a plate value, removes non-alphanumeric characters, converts the result to uppercase, searches the SQLite database, and writes a history record when at least one matching row exists.

The main architectural change is how the HTTP API is defined and how incoming data is described. FastAPI uses decorators such as `@app.get` and `@app.post` to declare endpoints. The Pydantic model `PlateRequest` describes the expected JSON request body, and FastAPI uses that type information for validation and API documentation [1][2].

This is useful for an API-oriented project, where a separate client such as test.py sends requests and needs a predictable contract.

Key point: FastAPI provides useful API-development features, but choosing FastAPI does not automatically make this particular database operation faster. The current endpoint uses a synchronous SQL interface and synchronous `def` handlers; performance depends on the complete application, database access, server configuration, and workload.

2. Project Structure

The project contains the application code and SQLite database. The testing script is kept in another directory. The tree below uses `tests/` as an illustrative directory name; replace it with the actual name if yours differs.

```
project/
|-- app.py
|-- fiftyville.db
`-- tests/
    `-- test.py
```

2.1 app.py

The main application module. It creates the FastAPI instance, opens the CS50 SQL interface, defines the Pydantic request model, registers GET and POST endpoints, and implements plate normalization.

2.2 fiftyville.db

The SQLite database file. The code reads from `bakery_security_logs` and writes to `history`. The database must exist at the expected path, and both tables must contain the columns referenced by the SQL statements.

2.3 test.py

This separate client script sends HTTP requests to the running FastAPI process. It can live outside the project directory because the communication occurs over HTTP. It does not need to import app.py, but it must use the correct host, port, path, HTTP method, and request body.

3. Why the Group Migrated from Flask to FastAPI

Both Flask and FastAPI can build web APIs. Flask is a lightweight WSGI web framework designed to be quick to start and flexible as an application grows [4]. FastAPI is designed around type-annotated API declarations, request validation, OpenAPI, and generated interactive documentation [1][2]. The migration is best understood as choosing tools that align with an API-first project, not as a claim that Flask is incapable of building APIs.

Area	Flask	FastAPI
Route declaration	Commonly uses @app.route with method options.	Uses clear operation decorators such as @app.get and @app.post.
Request data	Developers commonly access request data manually or add validation libraries.	A Pydantic model can describe and validate a JSON body directly.
Type hints	Useful to Python tooling, but not the central request-processing mechanism.	Type annotations are used for parsing, validation, error responses, and documentation.
API documentation	Usually configured or added with extensions/tools.	Generates OpenAPI schema and interactive docs automatically.
Async support	Modern Flask supports async views, but retains a WSGI-oriented design and related constraints.	ASGI-based and designed for async endpoints, while also supporting ordinary def functions.

Area	Flask	FastAPI
Flexibility	Minimal core; developers choose many components and conventions.	More API behavior is inferred from endpoint signatures and model declarations.

3.1 Advantages relevant to this project

- **Structured request schema:** PlateRequest documents that the request body must contain a string field named plate.
- **Automatic validation:** If the JSON body is missing plate or provides a value that cannot be validated as a string, FastAPI normally rejects the request with a 422 validation response before the endpoint logic runs [2].
- **Interactive documentation:** With the default setup, the team can open /docs to try endpoints in a browser, and /redoc to view alternative API documentation [1].
- **OpenAPI integration:** FastAPI generates a machine-readable API schema, which can help clients inspect endpoints or generate client code [1].
- **Type-friendly development:** Type annotations make the intended data shape clearer and provide better editor assistance [2].
- **Concurrency options:** FastAPI supports async endpoints, which can help for I/O-heavy workloads when the libraries used in the endpoint support asynchronous operations [3].

3.2 Important trade-offs

Flask remains a sound option for small services and traditional web applications. FastAPI introduces concepts such as Pydantic models, ASGI servers, and generated schemas. Async capabilities are not automatically applied to every operation, and synchronous database calls can still block the thread executing them. For this code, the clearest immediate benefits are request validation, endpoint declarations, and API documentation.

4. Imports and Application Initialization

```
from time import strftime
from cs50 import SQL

from fastapi import FastAPI
from fastapi.responses import PlainTextResponse
from pydantic import BaseModel
```

- **strftime:** Formats the current date and time for the history record.
- **SQL from cs50:** Provides the database interface used to execute SQL statements.
- **FastAPI:** Creates the API application instance.
- **PlainTextResponse:** Tells FastAPI that endpoint responses should be returned as plain text rather than JSON.
- **BaseModel:** The Pydantic base class used to define a structured request model.

4.1 Create the API application

```
app = FastAPI()
```

This creates the FastAPI application object. The server and framework use this object to locate registered endpoint functions and process incoming requests.

4.2 Connect to the database

```
db = SQL("sqlite:///fiftyville.db")
```

This creates the CS50 SQL interface for the SQLite database. The path is relative, so the program should normally be started from the directory where fiftyville.db is expected, or the project should use a deliberate configured path. A deployed application would also consider configuration and database lifecycle management.

5. The Pydantic Request Model

```
class PlateRequest(BaseModel):
    plate: str
```

This class defines the expected shape of the incoming JSON request body. A valid request body could look like:

```
{
    "plate": " ab-123 cd "
}
```

When a POST request reaches an endpoint whose parameter is annotated as `data: PlateRequest`, FastAPI reads the body, parses the JSON, and asks Pydantic to validate it against the model. The field annotation `plate: str` states that the field should be a string [2].

This improves the API boundary: invalid or missing body data is handled by the framework before the endpoint main logic runs. It does not validate whether the plate exists in the database; that business-rule check happens later in the function.

The current model does not specify constraints such as minimum length, maximum length, or a plate-number pattern. Those could be added if the project defines such rules.

6. GET Endpoint

```
@app.get("/", response_class=PlainTextResponse)
def index():
    return "Todo"
```

The `@app.get` decorator registers a handler for GET requests to the root path /. When a client makes a GET request to the server root, FastAPI calls index() and returns the text Todo.

The `response_class=PlainTextResponse` setting makes the response plain text. The endpoint is a placeholder because its response does not yet provide application status, a web interface, or license plate results.

GET and POST endpoints both use the root path /, but they are distinguished by their HTTP methods. This is valid API design: GET / and POST / can perform different operations.

7. POST Endpoint and Main Verification Logic

```
@app.post("/", response_class=PlainTextResponse)
def checking(data: PlateRequest):
    data.plate = remove(data.plate)

    check = db.execute(
        "SELECT COUNT(license_plate) as n "
        "FROM bakery_security_logs WHERE license_plate = ?",
        data.plate
    )

    if check[0]["n"]:
        db.execute(
            "INSERT INTO history(plate, direction, time) VALUES(?, ?, ?)",
            data.plate,
            "in",
            strftime("%Y/%m/%d %H:%M:%S")
        )
        return "True"

    return "False"
```

7.1 Receive and normalize the submitted plate

The parameter `data: PlateRequest` tells FastAPI to parse the JSON body into a PlateRequest model. The following line then replaces the field value with its cleaned version:

```
data.plate = remove(data.plate)
```

With the standard Pydantic configuration, model fields are mutable by default. A potentially clearer alternative is to keep the incoming model unchanged and use a local variable such as `plate = remove(data.plate)`, then use plate in the query and insert.

7.2 Query the database

```
check = db.execute(
    "SELECT COUNT(license_plate) as n "
    "FROM bakery_security_logs WHERE license_plate = ?",
    data.plate
)
```

The query counts matching non-NULL license_plate values in bakery_security_logs. The question-mark placeholder receives the plate as a parameter. Parameterized queries help prevent SQL injection by keeping the input separate from the SQL statement.

The CS50 SQL interface returns rows in a list-like structure, so `check[0]["n"]` accesses the count from the first returned row. The count is zero if there are no matching rows.

7.3 Branch on the result

The condition `if check[0]["n"]:` is true when the count is greater than zero. The application then inserts a row into history. If the count is zero, the insert is skipped and the function returns `"False"`.

7.4 Insert a history record

The INSERT statement writes three values to history:

- **plate:** The cleaned plate string.
- **direction:** The hard-coded value "in".
- **time:** The time when the application processes this request, formatted with strftime.

```
strftime("%Y/%m/%d %H:%M:%S")
```

The timestamp represents application processing time, not necessarily the time the vehicle entered the location. The code also does not infer actual travel direction from the security logs; it always writes "in" when a match exists.

7.5 Plain-text response

Because the endpoint declares PlainTextResponse, the returned strings True and False are delivered as plain text, not JSON booleans. A more explicit API could return structured JSON such as {"found": true} and suitable status codes, but changing the response contract would also require updating test.py or other clients.

8. The remove() Function

```
def remove(plate: str) -> str:
    word = ""
    for i in plate:
        if i.isalnum():
            word += i
    return word.upper()
```

The function accepts a string and returns a normalized version. The type annotations communicate that plate is expected to be a string and the returned result is also a string.

- **Initialize:** Start with an empty result string.
- **Iterate:** Examine each character of the submitted plate.
- **Filter:** Keep a character only when isalnum() returns True.
- **Build the result:** Append every accepted character.
- **Normalize case:** Convert the final string to uppercase.

Example input	Normalized output
ab-123	AB123
ab 123	AB123
xy_789!	XY789

This makes user-entered formatting more consistent. However, only the submitted input is normalized. If stored license_plate values have different formatting, the comparison may fail unless database values follow the same

convention.

9. End-to-End Request Lifecycle

```
test.py
   |
   | HTTP POST / with JSON body
   | {"plate": "ab-123"}
   v
FastAPI receives the request
   |
   v
PlateRequest validates the body
   |
   v
checking(data)
   |
   v
remove(data.plate)
   |
   v
SQLite query: count matching plates
   |
   +---- count > 0 ----> INSERT into history
   |                          |
   |                          v
   |                   plain-text "True"
   |
   +---- count = 0 ------------> plain-text "False"
```

The complete flow is:

- The group starts the FastAPI server.
- The separate test.py program sends an HTTP request to the configured URL.
- FastAPI matches the HTTP method and path to the endpoint.
- For POST requests, FastAPI parses and validates the JSON body as PlateRequest.
- checking() normalizes the plate using remove().
- The application queries fiftyville.db for matches.
- If at least one match exists, the application inserts a history record.
- FastAPI returns the resulting plain-text message to the client.

10. Running and Testing the FastAPI Application

10.1 Prepare the Python environment

Make sure the environment used by the group has FastAPI, Uvicorn, and the CS50 Python library installed. Use the package manager or environment workflow that your group actually uses. If test.py imports the third-party requests package, that package must also be available in the environment used to run the test script.

10.2 Start the server

```
uvicorn app:app --reload
```

In this command, the first app refers to the Python module app.py, and the second app refers to the FastAPI object created with `app = FastAPI()`. The --reload option is intended for development and restarts the server when source files change; it is not normally used for production deployment.

The server prints the local address at which it is listening. The actual URL and port depend on the command and configuration.

10.3 Interactive API documentation

With the default FastAPI configuration, open `/docs` on the server base URL to inspect and call endpoints through Swagger UI. `/redoc` provides an alternative documentation view. These pages are generated from the registered routes and request model [1].

10.4 Example request

A client can send a POST request to the root path with a JSON body. For example, curl can be used from a terminal:

```
curl -X POST "http://127.0.0.1:8000/" \
  -H "Content-Type: application/json" \
  -d '{"plate":"ab-123"}'
```

This example assumes Uvicorn is running locally on port 8000. If the submitted plate is found, the response body should be plain text True and a history row should be inserted. If no match is found, the response body should be False.

If test.py uses the Python requests library, it should send JSON rather than URL query parameters. For example, use `requests.post(url, json={"plate": "ab-123"})`.

11. Design Notes and Potential Improvements

11.1 The current handlers are synchronous

Both endpoint functions use normal def rather than async def. FastAPI supports both styles; normal def path operations are run in an external thread pool by the framework [3]. This is a reasonable option when calling a synchronous library such as the current CS50 SQL interface. Merely changing def to async def would not make the synchronous database call asynchronous.

11.2 A match means only that a plate exists in the table

The query checks the entire bakery_security_logs table for a matching plate. It does not limit the match to a specific time, event type, or entrance. If the intended behavior is to verify a particular entry event, the query must reflect those project requirements.

11.3 Repeated POST requests produce repeated history rows

Every successful request creates a new history record, even if the same plate was already submitted. If the desired behavior is one record per security event, the application needs a rule for identifying that event and avoiding accidental duplicate records.

11.4 Validate business rules too

Pydantic checks the request shape and types, but a string field can still be empty or consist entirely of punctuation. The project may want to reject a plate when normalization produces an empty string, or validate length and accepted formats based on the real requirements.

11.5 Return structured responses

Plain-text True and False are easy to test, but JSON responses such as {"found": true} are easier to extend with additional fields. If the response format changes, update test.py and document the API contract.

11.6 Use a descriptive function name

The endpoint handler name checking() is valid, but a name such as verify_plate() would more clearly communicate its purpose.

11.7 Keep configuration predictable

The SQLite URL uses a relative path. Starting the application from an unexpected working directory can cause it to look for the database somewhere else. Document the expected launch directory or configure a path explicitly.

11.8 Keep development and production settings separate

Use --reload while developing and testing. For deployment, use an appropriate server configuration, secure configuration values, and suitable access controls for the service and database.

12. Conclusion

The migration from Flask to FastAPI changes the API framework and introduces a clearer request model, automatic request validation, and generated documentation. The core license plate logic remains the same: normalize the submitted plate, count matching database rows, and record a history row if at least one match exists.

For this project, the most immediate benefits of FastAPI are its Pydantic request model, type-driven validation, clear GET and POST decorators, and automatically generated API documentation. Flask could also support the same application, but some of these features would typically require additional design decisions or extensions.

The migration is not a complete change to the application business logic. It does not by itself make the existing synchronous database call non-blocking or guarantee better measured performance. The next steps should focus on validating real plate formats, ensuring database values are comparable, defining the intended meaning of a positive match, and using consistent response data.

13. Official References

[1] FastAPI, “Features” - OpenAPI integration, generated interactive documentation, and related functionality.
https://fastapi.tiangolo.com/features/

[2] FastAPI, “Python Types” - use of type annotations for parsing, validation, errors, and documentation.
https://fastapi.tiangolo.com/python-types/

[3] FastAPI, “Concurrency and async / await” - how synchronous and asynchronous path operation functions are handled.
https://fastapi.tiangolo.com/async/

[4] Flask, “Welcome to Flask” - Flask as a lightweight WSGI framework and its general design philosophy.
https://flask.palletsprojects.com/en/stable/

This report describes the FastAPI code supplied for this version of the project. The actual contents of test.py and the database schema were not provided, so details about those parts are described at the interface level rather than assumed.